Geometric and Learned Relay Control for Connectivity-Constrained Drone Delivery in Disaster Response

Gwang-Jong Ko[a], Taesu Cheong[a,*]

[a]*School of Industrial and Management Engineering, Korea University, 145, Anam-ro, Seongbuk-gu, Seoul, 02841, Republic of Korea*

Abstract

In disaster relief, delivery drones must stay connected to a control centre through a multi-hop relay chain formed by the fleet itself, while carrying supplies with limited capacity and returning to reload. We model this as a makespan-minimisation problem in which every drone can either deliver or relay, and connectivity is a hard constraint enforced at every time step. We propose two controllers that share the same connectivity-preserving projection: a geometric chain-relay heuristic with a deadlock-escape mechanism, and a fleet-size-agnostic policy learned once with discrete soft actor-critic over a set-based, autoregressive manager. Both are compared on identical instances with a paired protocol across fleets of 3 to 12 drones and 20 to 400 destinations, against four learning-free baselines and a time-expanded mixed-integer reference model on small instances. The learned policy completes every instance in every configuration; the heuristic completes 98–100% with the escape mechanism and 62% without it. The two controllers divide the configuration space: with three drones the heuristic is 30–39 steps faster in paired median, with four drones the two tie over 480 rollouts, and with five or more drones the learned policy is faster by 38 to 178 steps; the policy is trained only on three and four drones and generalises to every larger fleet without retraining. The three-drone deficit is localised to a single decision, which drone relays, and ten policy-side interventions covering information, credit assignment, decision structure, initialisation and reward shaping all fail to remove it; half of it disappears when the connectivity projection is changed from radial shrinking to tangential sliding, showing that the deficit is partly a property of how the constraint is enforced rather than of the learner. Both controllers run in milliseconds per decision, three orders of magnitude below exact optimisation, which does not return a plan for ten destinations within ninety minutes.

Keywords: drone delivery, relay network, connectivity constraint, multi-agent reinforcement learning, heuristic, disaster logistics

1. Introduction

When communication, logistics and road infrastructure are destroyed, a fleet of drones can carry emergency supplies from a temporary control centre to many destinations. Two features make this different from parcel delivery. First, every drone must be reachable by the control centre at all times, because loading and unloading are supervised remotely and the delivery status must be tracked; with the ground network gone, reachability is provided only by the drones themselves acting as radio relays in a multi-hop chain. Second, the fleet is small and each drone carries a few units, so drones shuttle between the centre and the destinations and every drone that stands still as a relay is a drone that is not delivering. The operational question is how to assign delivery and relay roles over time so that the last destination is served as early as possible without ever breaking the chain.

Three properties make the problem hard. A hard connectivity constraint creates deadlocks: a greedy assignment that sends every drone to its nearest destination completes none of the standard instances in our study, because the fleet pulls itself apart until no drone can move. Relaying competes directly with throughput, so the right number and identity of relays changes with the geometry of the remaining destinations and with how many drones are left with cargo. And exact optimisation does not scale: a time-expanded mixed-integer model on a 98-metre lattice returns plans for four and six destinations but no feasible plan for ten within ninety minutes.

Proposal. We propose two controllers for this problem and study them side by side.

1. A *geometric chain-relay heuristic*. The drone farthest from the centre with cargo is the lead; the number of relays is the number of communication hops its target needs; the relays are the next-farthest drones, stationed on the segment from the centre to the lead at intervals of $0.9R$. Because the rule is deterministic it can repeat a deadlocked assignment indefinitely, so we add a *deadlock-escape* mechanism that swaps the lead whenever the fleet has stalled for a fixed number of steps. The escape mechanism alone lifts completion on the standard configuration from 62% to 98%.
2. A *learned policy* that is agnostic to fleet size. A manager assigns each drone one of seven discrete options every twenty steps; it encodes the drones, candidate destinations and relay slot as a set, pools them with attention, and decides autoregressively so that a formation of one lead and

*Corresponding author.

Email addresses: koptimizer@korea.ac.kr (Gwang-Jong Ko), tcheong@korea.ac.kr (Taesu Cheong)

k relays can be composed deliberately. It is trained once with discrete soft actor-critic on fleets of three and four drones and then run without retraining on fleets of up to twelve.

Both controllers act through the same connectivity-preserving projection: each commanded move is scaled by the largest factor that keeps the fleet connected. Connectivity is therefore guaranteed by construction for both, and the empirical question is whether a controller makes progress under the constraint rather than whether it violates it.

Research questions. We organise the study around four questions.

RQ1 Do the proposed controllers always complete the delivery while connectivity is enforced at every step, and when they do not, why not?

RQ2 On small instances, how close are the proposed controllers to a reference plan obtained by exact optimisation of a lattice model?

RQ3 How does the performance of the two controllers differ across environments, in particular across fleet size, number of destinations, depot position and communication radius; does a policy trained on small fleets generalise to larger ones; and where do the differences come from?

RQ4 Are the components of each controller necessary?

Findings. The learned policy completes every instance in every configuration tested; the heuristic completes 98–100% with the escape mechanism (RQ1). On instances small enough for the lattice model, both controllers are within a few percent of the reference plan, and the reference plan itself is a restriction rather than a bound of the continuous problem (RQ2). The two controllers divide the configuration space by fleet size: the heuristic is faster with three drones, the two tie with four, and the learned policy is faster with five or more, by 62 to 178 steps up to ten drones and 76 at twelve, although it never saw more than four drones in training; the three-drone deficit is a single decision, which drone relays, and it resists ten policy-side interventions but shrinks by half when the connectivity projection is changed (RQ3). Removing rule regularisation, configuration randomisation or entropy annealing from the learned policy costs hundreds of steps or completion, and removing the escape mechanism from the heuristic costs a third of its completions (RQ4).

Contributions. (i) A geometric chain-relay heuristic with deadlock escape and a fleet-size-agnostic learned policy for connectivity-constrained delivery, both built on a projection that makes the constraint hold by construction. (ii) A paired evaluation across fleet sizes that shows the two controllers are complementary, together with the mechanism behind the boundary. (iii) A set of ten documented negative results on closing the scarce-fleet deficit by changing the learner, and a sensitivity result showing that half of the deficit is attributable to the enforcement model. (iv) A lattice MILP reference with an honest account of what it does and does not bound.

2. Related Work

Drone delivery routing. Routing with drones has been studied since the flying-sidekick travelling salesman problem [1] and its multi-drone extensions [2, 3, 4]. These models treat communication as free: a drone is never required to be reachable, and no vehicle gives up its delivery role to serve as a relay. Learned solvers for min-max multi-vehicle routing [5, 6] show that set-based policies generalise across problem sizes, which is the property we need across fleet sizes, but again without a connectivity constraint. Dual-mission fleets in which the same drones deliver and sense [7] formalise a per-delivery detour budget; the secondary task there is sensing, not communication.

UAV relays and connectivity maintenance. Aerial relay positioning is its own literature [8, 9, 10], and joint optimisation of mission and relay UAVs with multi-agent learning exists [11, 12]. In these works the relays are the mission, or a dedicated sub-fleet with its own reward; they do not carry cargo, do not reload, and do not minimise a delivery makespan. Connectivity maintenance for multi-robot navigation has been approached with learned controllers [13]. Switching between a task role and a relay role is an established idea in multi-robot exploration [14, 15], where relay robots land and act as fixed nodes.

Where this work sits. To the best of our knowledge, no prior work combines (i) a homogeneous fleet carrying physical cargo to many destinations with capacity and reloading, (ii) a hard requirement that every drone be connected to the centre by a multi-hop chain at every step, and (iii) a makespan objective. In this setting relay duty competes with cargo throughput and the fleet must return to the depot repeatedly, which is what makes role assignment a sequential decision rather than a one-off placement.

Learning machinery. On the learning side we build on soft actor-critic [16], centralised critics for decentralised execution [17, 18], value decomposition [19, 20], options as a semi-Markov decision process [21], invalid-action masking [22] and safe-action projection [23]. Our negative results in Section 6.3 test several of these ideas directly: per-agent credit, joint-action critics, monotonic value mixing and potential-style shaping [24] are all tried as remedies for the scarce-fleet deficit and none of them removes it.

3. Problem Statement

A control centre sits at $c \in [0, 1000]^2$ metres and M destinations $d_1, \ldots, d_M$ lie within $0.9\,NR$ of it, where N is the number of drones and $R = 300$ m the communication range. Each drone flies at $v = 13.89$ m/s and carries $Q = 5$ units. Every delivery and every reload incurs a *service delay*: unloading one unit at a destination takes $S = 10$ steps and reloading to Q at the centre takes S steps, and during these steps the drone is stationary. A destination or the centre is reached when a drone is within 25 m of it, and one step is one second. The service delays matter

for the connectivity constraint below: a stationary drone cannot move to keep a link alive, so its teammates must keep the chain intact around it. Every destination needs exactly one unit. Drones do not collide, are not affected by wind, and battery is not modelled.

Connectivity. Let G_t be the graph whose vertices are the centre and the N drones and whose edges join vertices within R of each other. The fleet is *connected* at step t if G_t is connected, which is the same as saying that every drone has a multi-hop path to the centre. The constraint is that G_t is connected at every t. It is enforced rather than penalised (Section 4.1).

Objective. The makespan is the step at which the last destination is served. A controller *completes* an instance if it serves all destinations within a budget of 10,000 steps. We report completion rate and, over completed instances, makespan.

Configurations. A configuration is a triple of fleet size N, number of destinations M and depot placement (fixed at the map centre, or uniformly random). The *standard* configuration is $N = 4$, $M = 50$ with the fixed depot. The learned policy is trained with $N \in \{3, 4\}$, $M \in [20, 80]$ and a random depot; all other configurations are unseen by it. The heuristic has no training.

3.1. A space–time network model, exact on the discretisation

The third element we propose, besides the two controllers, is an optimisation model of the problem. It serves two purposes: it states the problem without ambiguity, and on small instances it provides a plan obtained by exact optimisation against which both controllers are measured.

Why the model takes this form. Vehicle-routing formulations of drone delivery [1, 3] assign routes to vehicles and never need to know where a vehicle is at a given instant. The connectivity requirement of Section 3 is different in kind: it is a constraint on the *positions of all drones at every step*, because the communication graph G_t is built from those positions. A model must therefore carry every drone's position over time, and the natural object for that is a *time-expanded network* [25]: the set of positions is copied once per time period and arcs join a position in one period to the positions reachable in the next. Positions in the plane are uncountable, so space is discretised as well, to a lattice; the result is a *space–time network* on which drones are unit flows, and a time-indexed MILP on this network is exact for the discretised problem. Connectivity is then a property of which lattice nodes are occupied in each period, and we encode it with a single-commodity flow per period that is feasible exactly when the occupied nodes form a multi-hop chain to the centre. The model is *exact* in the sense that its optimum is the true optimum of the discretised problem; its relation to the continuous problem is stated at the end of this section.

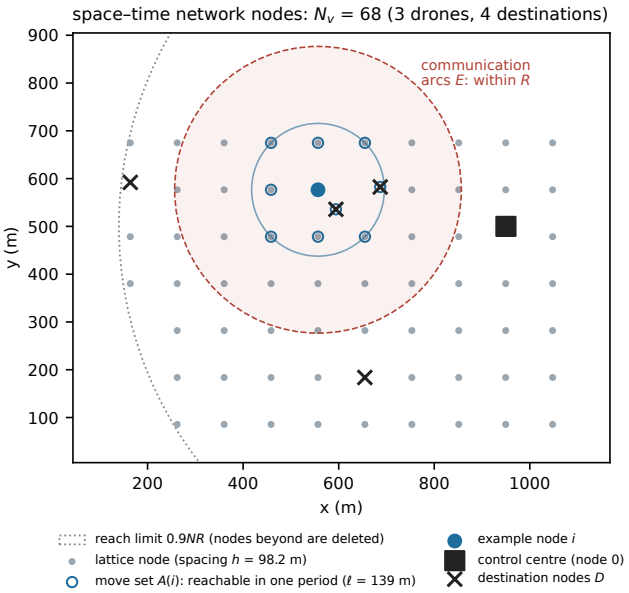


Figure 1: Nodes of the space–time network for a reference instance with three drones and four destinations: the centre (node 0), the destination nodes D, and the lattice G of spacing $h = 98.2$ m clipped to the reach limit $0.9NR$. For one example node i the figure marks its move set $A(i)$ (the nodes reachable in one period, $\ell = 138.9$ m) and its communication disc of radius R, which defines its arcs in E.

Discretisation. A period is $\kappa = S = 10$ steps, so a service occupies exactly one period and a manager decision of the learned policy spans two; the per-period flight range is $\ell = v\kappa = 138.9$ m. Space is a square lattice G of spacing $h = \ell/\sqrt{2} = 98.2$ m over the bounding box of the instance, from which nodes farther than $0.9NR$ from the centre are deleted because no chain can reach them; the centre and every destination are added as exact nodes, so the interaction test is exact there. Figure 1 shows the resulting node set for a reference instance. With this spacing all eight king moves are feasible in one period; the move set and the communication set are

$$A(i) = \{j \in N_v : \|p_i - p_j\| \le \ell\} \ni i, \tag{1}$$

$$E = \{(i, j) \in N_v^2 : i \ne j,\ \|p_i - p_j\| \le R\}. \tag{2}$$

Axis-aligned moves cover h rather than ℓ per period, so the lattice slows a drone by up to 29% in those directions; this discretisation error always acts against the model. The lattice must also be fine enough to hold a relay chain: serving a destination at distance d needs the intermediate drones placed within the chain's slack $NR - d$, which requires roughly $h \lesssim (NR - d_{\max})/(N - 1)$; a 196 m lattice makes the reference instances infeasible although they are feasible in the plane, and 98 m does not.

Notation. Table 1 lists the sets, parameters and decision variables. Drones are identical, so a drone index only distinguishes

units; the symmetry-breaking cut at the end of the model exploits this. The model is given in (3)–(16).

$$\min \; C + \varepsilon \sum_{i \in N_v \setminus \{0\}} \sum_{k \in K} \sum_{t \in T} y_{ikt} \tag{3}$$

$$\text{s.t.} \; \sum_{i \in N_v} y_{ikt} = 1 \qquad \forall k \in K,\ t \in T \tag{4}$$

$$y_{0k0} = 1 \qquad \forall k \in K \tag{5}$$

$$y_{i,k,t+1} \le \sum_{j \in A(i)} y_{jkt} \qquad \forall i \in N_v,\ k \in K,\ t < \bar{T} \tag{6}$$

$$\sum_{k \in K} \sum_{t < \bar{T}} s_{jkt} = 1 \qquad \forall j \in D \tag{7}$$

$$s_{jkt} \le y_{jkt}, \qquad s_{jkt} \le y_{j,k,t+1} \qquad \forall j \in D,\ k \in K,\ t < \bar{T} \tag{8}$$

$$r_{kt} \le y_{0kt}, \qquad r_{kt} \le y_{0,k,t+1} \qquad \forall k \in K,\ t < \bar{T} \tag{9}$$

$$\sum_{j \in D} s_{jkt} + r_{kt} \le 1, \qquad \sum_{j \in D} s_{jkt} \le q_{kt} \qquad \forall k \in K,\ t < \bar{T} \tag{10}$$

$$q_{k,t+1} = q_{kt} - \sum_{j \in D} s_{jkt} + \rho_{kt}, \qquad q_{k0} = Q \qquad \forall k \in K,\ t < \bar{T} \tag{11}$$

$$\rho_{kt} \le Q\, r_{kt}, \qquad q_{k,t+1} \ge Q\, r_{kt} \qquad \forall k \in K,\ t < \bar{T} \tag{12}$$

$$\sum_{j:(i,j) \in E} g_{ijt} - \sum_{j:(j,i) \in E} g_{jit} = u_{it} \qquad \forall i \in N_v \setminus \{0\},\ t \in T \tag{13}$$

$$g_{ijt} \le N\, u_{it} \qquad \forall (i,j) \in E,\ t \in T \tag{14}$$

$$C \ge t\, s_{jkt} \qquad \forall j \in D,\ k \in K,\ t < \bar{T} \tag{15}$$

$$s_{jkt} = 0 \qquad \forall k \in K,\ j \in D \text{ with } \mathrm{idx}(j) < k,\ t < \bar{T} \tag{16}$$

Reading the model. The objective (3) minimises the makespan C; the second term is a tie-break with weight $\varepsilon \bar{T} N \ll 1$ that prefers plans keeping idle drones at the centre and cannot change the optimal makespan.

Position and movement, (4)–(6). Every drone is at exactly one node at every period boundary and starts at the centre. A drone may be at node i at boundary $t+1$ only if at boundary t it was at i itself or at a node from which i is reachable within one period. Because every move takes exactly one period, the position of every drone is defined at every boundary, which is what the connectivity constraints below need.

Deliveries, cargo and reloading, (7)–(12). Each destination is served exactly once (7). A delivery during period t requires the drone to be at the destination at both ends of the period (8), which is the service delay of Section 3: the drone stands still for one period. Reloading is likewise a full period at the centre (9). A drone does at most one service per period and can only deliver what it carries (10); cargo is tracked by (11) starting full; and a reload fills the drone completely (12), because ρ_{kt} can be positive only in a reload period and the next cargo must then be Q.

Multi-hop connectivity, (13)–(14). These two families are the part that distinguishes this model from drone-routing models. In every period, every occupied node ships one unit of flow, the centre is the only sink, and an arc may carry flow only if it leaves an occupied node. The system is feasible if and only if every occupied node is joined to the centre by a path whose intermediate nodes are occupied and whose consecutive distances are at most R, that is, if and only if the simulator's multi-hop connectivity test succeeds at that period boundary. The proof is short: if such paths exist, routing each node's unit along its path satisfies both families; conversely an unoccupied node $i \neq 0$ has $g_{ijt} = 0$ on all its outgoing arcs by (14) and therefore no incoming flow by (13), so all flow travels inside the occupied set and, since the centre is the only node without a conservation row, a flow decomposition yields the required paths. The mirror-image capacity rows $g_{ijt} \le N u_{jt}$ are implied and omitted, which halves the dominant row count. Note what the constraint does to the problem: a drone that has nothing to deliver cannot simply wait at the centre if a teammate needs it as a hop, and a drone cannot fly to a destination that would leave it farther than R from every occupied node. Relaying is therefore not a separate task but a consequence of where the fleet is.

Makespan and symmetry, (15)–(16). The makespan is at least the period of every delivery. Because drones are identical, any plan can be relabelled so that drone k serves no destination with index below k; (16) imposes this ordering and removes no objective value.

What the optimum means. The optimum of the model is exact for the discretised problem. For the continuous problem of Sec-

Table 1: Notation of the space–time network model.

Symbol	Domain	Meaning
Sets		
$N_v = \{0\} \cup D \cup G$		nodes: the control centre (index 0), the destination nodes D, the lattice nodes G
K	$\lvert K\rvert = N$	drones
$T = \{0, 1, \ldots, \bar{T}\}$		period boundaries; period t runs from boundary t to $t + 1$
$A(i) \subseteq N_v$		nodes reachable from i within one period, including i (move set)
$E \subseteq N_v^2$		ordered pairs of distinct nodes within communication range R (communication arcs)
Parameters		
Q	5	cargo capacity (units)
R	300 m	communication range
κ, ℓ, h	10 steps, 138.9 m, 98.2 m	period length, flight range per period, lattice spacing
$\bar{T}$		horizon in periods, set to 1.15× the heuristic's makespan
ε	10^{-5}	tie-breaking weight
$\mathrm{idx}(j)$		index of destination j in a fixed ordering of D
Decision variables		
y_{ikt}	$\{0, 1\}$	1 if drone k is at node i at boundary t
s_{jkt}	$\{0, 1\}$	1 if drone k delivers to destination j during period t
r_{kt}	$\{0, 1\}$	1 if drone k reloads at the centre during period t
q_{kt}	$\{0, \ldots, Q\}$	cargo on board drone k at boundary t
ρ_{kt}	$[0, Q]$	cargo taken on by drone k during period t
g_{ijt}	$[0, N]$	connectivity flow on arc (i, j) in period t
u_{it}	$\{0, \ldots, N\}$	occupancy of node i at boundary t, $u_{it} = \sum_k y_{ikt}$, with $u_{0t} := 1$
C	$[0, \bar{T}]$	makespan in periods

tion 3 it is an *upper bound*: drones are confined to lattice nodes and axis moves are shortened, so every lattice plan is a feasible continuous plan but not conversely, and a continuous controller can therefore be faster than the lattice optimum. The solver's dual bound is a bound on the lattice problem only. A certified lower bound through a cell relaxation, in which a lattice node stands for its cell and communication is tested by the minimum distance between cells, was attempted and abandoned: it inflates a three-hop chain's reach from 900 m to 1,317 m and returns bounds below the trivial analytic bound. We therefore call the solver's value a *reference plan*, replay every plan in the continuous simulator, and report the replayed makespan.

4. Proposed Controllers

Figure 2 summarises the setting and the two controllers. Both controllers decide at the level of *goals*: at every decision point each drone is given a goal (a destination, the centre, or a relay slot) and flies towards it in a straight line at full speed until the next decision point. The two controllers differ only in how goals are chosen; the option set, the low-level flight, the relay-slot geometry and the connectivity projection are shared and are described first.

4.1. Shared interface: options, relay slots, projection

Decision points and options. A decision point occurs every 20 steps, and additionally as soon as any drone's goal becomes invalid: its destination has been served by another drone, it has run out of cargo while heading to a destination, it has become full while heading to the centre, or its relay slot has disappeared because the chain it belonged to shortened. At a decision point every drone receives one of $K = 7$ options: one of its five *candidate destinations*, which are the five unserved destinations nearest to it; *return* to the centre; or *relay*. An option is *valid* for a drone only if it makes sense at that moment: a destination option requires cargo on board and an existing candidate, return requires the drone not to be full, and relay requires a free relay slot (defined next) and at least one teammate that is not itself relaying. Invalid options are masked for the learned policy and never produced by the heuristic; a drone with no valid option returns to the centre.

Relay slots. The environment, not the controller, decides where a relaying drone stands. For drone i the *frontier* f_i is the point farthest from the centre among the delivery goals of the other drones and the current positions of the other non-relaying drones (if there is none, the farthest unserved destination). A chain to f_i at distance d needs $n = \min(\lceil d/(0.9R)\rceil - 1,\ N - 1)$ relay slots, placed on the segment from the centre to f_i at the fractions $1/(n + 1), \ldots, n/(n + 1)$ of its length; the factor 0.9 leaves slack so that a drone standing slightly off its slot does not break the link. Drone i takes the free slot nearest to it. A drone whose slot disappears holds position until the next decision point. The controllers therefore decide only *whether* a drone relays; the slot follows from the fleet's geometry.

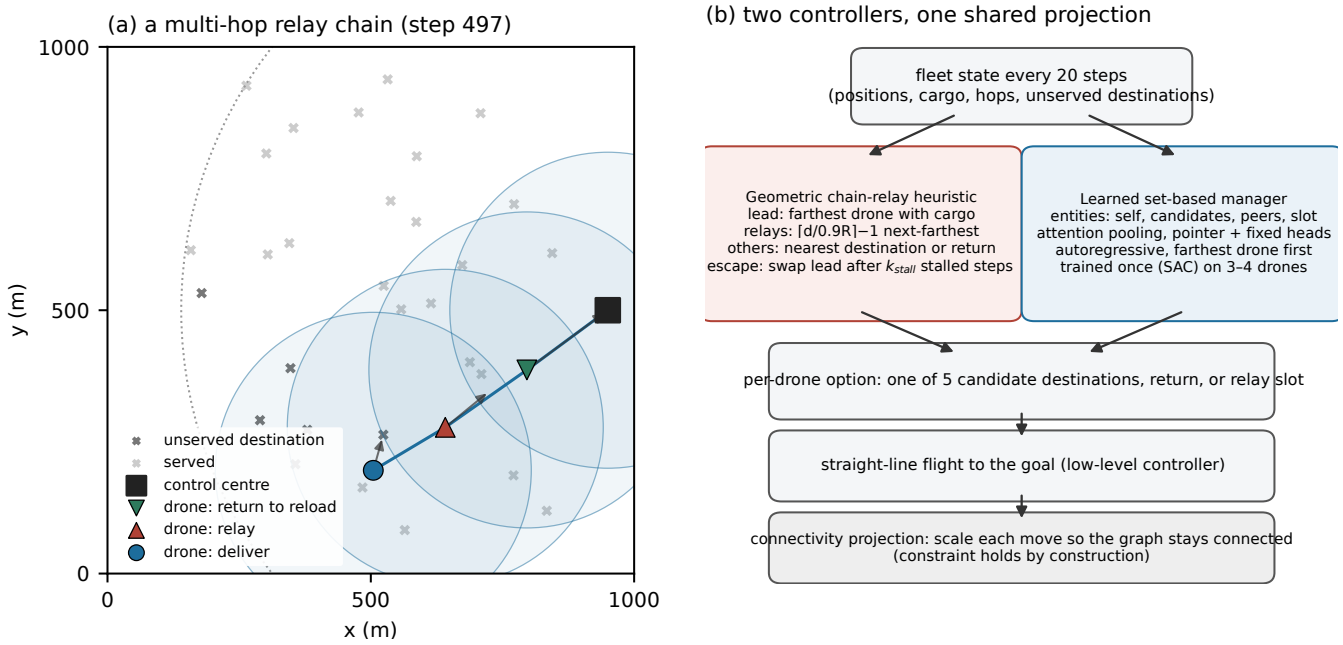


Figure 2: (a) An instance with three drones under the heuristic at step 497: the lead (blue circle) heads for a far destination, one drone relays on the segment to the centre (red triangle), and one drone returns to the centre to reload (green triangle); shaded discs are communication ranges and the dotted circle is the reachable region $0.9NR$. (b) The two controllers emit the same per-drone options and share the straight-line low-level controller and the connectivity projection.

Low-level flight and connectivity projection. Every step, each drone commands a displacement of length v towards its goal (zero while it is serving). Enforcing connectivity by a reward penalty would punish situations that no action can prevent, for example a drone that is stationary while unloading and is stranded when a teammate departs; connectivity is therefore enforced by projection. If the commanded positions keep G_t connected they are applied. Otherwise the projection repeatedly selects one moving drone, scales its displacement down to the largest factor in $[0, 1]$ under which the fleet is connected (found by bisection with the other drones' current scalings fixed), and stops as soon as the fleet is connected; at each round it selects the drone whose restriction leaves the most drones connected and breaks ties at random, so that no single anchor drone is starved. The previous positions are always connected, so zero scaling is a valid fallback and the constraint holds by construction. A drone whose displacement is scaled below 10% is *stalled* for that step, and a *partial deadlock* is a step in which all but at most one of the moving drones are stalled; the number of consecutive partial-deadlock steps, δ_t, is what the heuristic's escape mechanism monitors. This projection scales a displacement along its own direction; Section 6.3 studies a variant that keeps the tangential component.

4.2. Geometric chain-relay heuristic with deadlock escape

The heuristic is a deterministic rule evaluated at every decision point (Procedure 1). It first gives every drone a default: a drone with cargo goes to its nearest unserved destination and a drone without cargo returns to the centre. It then selects the *lead*, the drone with cargo that is farthest from the centre, computes how many hops the lead's target needs, and turns that many of the remaining drones, the farthest ones first, into relays. The number of relays is adaptive and the relays are the drones already closest to where the chain must stand, which is what makes the rule cheap: it costs $O(N \log N)$ per decision and 0.05–0.12 ms in our implementation.

Procedure 1. Geometric chain-relay heuristic with deadlock escape, run at every decision point. State kept between calls: remaining escape length e (initially 0) and escape count m (initially 0); both reset at the start of an episode. Parameters: $k_{\text{stall}} = 120$, $h_{\text{hold}} = 20$, mode $\in$ {lead swap, retreat}.

1. **Escape trigger.** If the fleet has been in partial deadlock for $\delta_t \geq k_{\text{stall}}$ consecutive steps and $e = 0$, set $e \leftarrow h_{\text{hold}}$ and $m \leftarrow m + 1$.
2. **Default assignment.** For each drone i: if it has no cargo, option $\leftarrow$ return; else if it has no unserved candidate, option $\leftarrow$ relay; else option $\leftarrow$ its nearest unserved destination.
3. **Lead.** If no destination is unserved, stop. Otherwise let $r_i = \|p_i - c\|$ and let the lead be the drone with cargo that maximises r_i (if no drone has cargo, the drone maximising r_i).
4. **Relay count.** Let $d = \max(r_{\text{lead}}, \|d_{\text{goal}} - c\|)$ where d_{goal} is the lead's nearest unserved destination, and $n = \min(\max(1, \lceil d/(0.9R) \rceil) - 1,\ N - 1)$.
5. **Relay identity.** Sort the drones other than the lead by r_i in decreasing order and set the option of the first n of them to relay.
6. **Escape manoeuvre** (only if $e > 0$; then $e \leftarrow e - 1$). *Lead swap*: sort all drones by r_i in decreasing order, let the new lead be the $\min(m, N - 1)$-th drone of that order (0-based), set the old lead's option to return and, if the new lead has cargo and an unserved candidate, set its option to its nearest unserved destination. *Retreat*: set the option of every drone except the lead to return.
7. Return the options; the environment converts them into goals and assigns relay slots.

Deadlock escape. Because the rule is deterministic, a geometry that stalls the fleet makes it re-issue the same assignment at every decision point and the stall persists; without an escape mechanism the rule completes 62% of standard instances. The

escape mechanism is a timed override. It *starts* at the first decision point at which the fleet has been in partial deadlock for at least k_{stall} consecutive steps and no escape is in progress. It *lasts* for h_{hold} decision points, that is, roughly $20\,h_{\text{hold}}$ steps, fewer when goals become invalid early; during these decision points the rule is evaluated as usual and then overridden as in step 6. It *ends* when the count e reaches zero, after which the plain rule resumes; if the deadlock persists it can start again at the next decision point. Two modes are implemented. In *lead swap* the current lead is sent back to the centre and the role of lead passes to another drone; on the m-th escape of an episode it passes to the m-th farthest drone, so that repeated escapes cycle through the fleet rather than swapping the same two drones. In *retreat* every drone except the lead returns towards the centre, which slackens the chain. Section 6.4 selects the mode and the two parameters on tuning seeds; lead swap with $k_{\text{stall}} = 120$ and $h_{\text{hold}} = 20$ is used in every comparison.

4.3. Hierarchical learned policy

The learned controller is hierarchical (Figure 3). The high level, a *manager*, acts only at decision points and outputs one option per drone; the low level is the shared straight-line flight of Section 4.1, so the manager's choices are held for 20 steps and the problem it solves is a semi-Markov decision process over options [21]. Only the manager is learned. Separating the levels has two consequences that the rest of the design relies on: the manager sees the fleet at a coarse time scale at which a formation can be held, and the low level, being a fixed rule, can be reasoned about and projected for connectivity without learning.

Set-based encoder. The manager encodes drone i from four kinds of entities, all distances divided by R. *Own state*, 10 features: cargo fraction; distance to the centre; hop depth divided by 8; fraction of destinations unserved; elapsed time divided by the budget; its realised-movement ratio (an exponential average of achieved over commanded displacement); whether it could advance one full step towards its goal alone without disconnecting the fleet; the fleet-wide realised-movement ratio; service timer; and whether it is a cut vertex of G_t. *Candidate destinations*, five entities of 6 features: distance; the two components of the unit direction vector; whether a teammate already targets it; its distance from the centre; and the hops its chain requires, divided by 8. *Peers*, $N - 1$ entities of 9 features: distance; the two components of the relative position vector; cargo fraction; hop depth divided by 8; and a four-way one-hot of the peer's current goal kind (deliver, return, hold, relay). *Relay slot*, 4 features: distance; the two components of the unit direction vector; and the number of slots in the current chain, divided by 8. Each entity type passes through a two-layer MLP of width 128. The drone's own embedding is the query of two multi-head attention layers (4 heads) that pool the peer set and the candidate set into one vector each; the four vectors are concatenated and fused by a further MLP into the drone's embedding h_i. A *pointer head* scores each candidate from $[h_i; e_{\text{cand}}]$ and a *fixed head* scores return and relay from h_i; the seven scores are masked and normalised by a softmax. Nothing in the network depends on N or M, which is why one set of weights runs on fleets it never saw.

Autoregressive decision. The N options are not sampled independently. Drones are ordered by distance from the centre, farthest first; the manager samples the first drone's option, writes its new role into the role one-hot of the peer features that the remaining drones see, re-encodes, samples the next drone, and so on (Procedure 2). A formation of one lead and k relays arises only by coincidence under independent sampling but can be composed deliberately under sequential decisions; the cost is N sequential network calls per decision point.

Training. The manager is trained with discrete soft actor-critic [16] on option transitions. A transition stores the fleet observation at a decision point, the N options, the option return $\sum_\tau \gamma^\tau r_\tau$ accumulated over the steps until the next decision point, the observation at that point, and the discount $\gamma^{\Delta t}$ with $\gamma = 0.997$ per step. The team reward is $+10$ per delivery, $+30$ per reload, $-0.01N$ per step and a residual penalty of $-0.1N$ times the fraction of destinations still unserved. A twin-Q critic with attention across the drone embeddings outputs per-drone action values whose mean over drones is regressed on the option return; the actor minimises the standard expectation form $\mathbb{E}_{a\sim\pi}[\alpha \log \pi(a) - Q(a)]$ per drone, with teacher forcing of the autoregressive order. Three ingredients of the recipe matter and are ablated in Section 6.4. (i) The entropy target is annealed linearly from $0.6 \log n_{\text{valid}}$ to $0.25 \log n_{\text{valid}}$ over the first 400 episodes: a high target keeps the policy random enough to escape deadlocks early in training, a low one lets it hold a multi-step formation later. (ii) A cross-entropy term with weight 2.0 pulls the policy towards the option the heuristic's assignment rule (Procedure 1, steps 2–5, without escape) would take, wherever that option is valid. (iii) Every training episode samples $N \in \{3, 4\}$, $M \in [20, 80]$ and a random depot. Optimisation uses Adam at 3×10^{-4}, batches of 256 transitions from a replay buffer of 60,000, one gradient step every 4 environment steps after a warm-up of 400 transitions, and Polyak averaging of the target critic with $\tau = 0.005$. Checkpoints are scored every 50 episodes on 80 held-out instances with a 3,000-step budget by $100f + 40(1 - \bar{t}/3000)$, where f is the completion rate and $\bar{t}$ the mean steps; training stops after eight evaluations without improvement or at 1,000 episodes. One run takes about 13 hours on a single RTX 5080 when four runs share the GPU.

5. Experimental Setup

Instances and configurations. Instances are generated from integer seeds; a seed, N, M and the depot mode determine the instance exactly, so two controllers can be run on identical instances. Evaluation seeds are disjoint from the seeds used for training, for checkpoint selection and for tuning the heuristic. We evaluate on the standard configuration ($N = 4$, $M = 50$, fixed depot) and on eight unseen configurations with random depots: $N/M \in \{3/20, 3/30, 4/80, 5/50, 6/80, 8/150, 10/300, 12/400\}$.

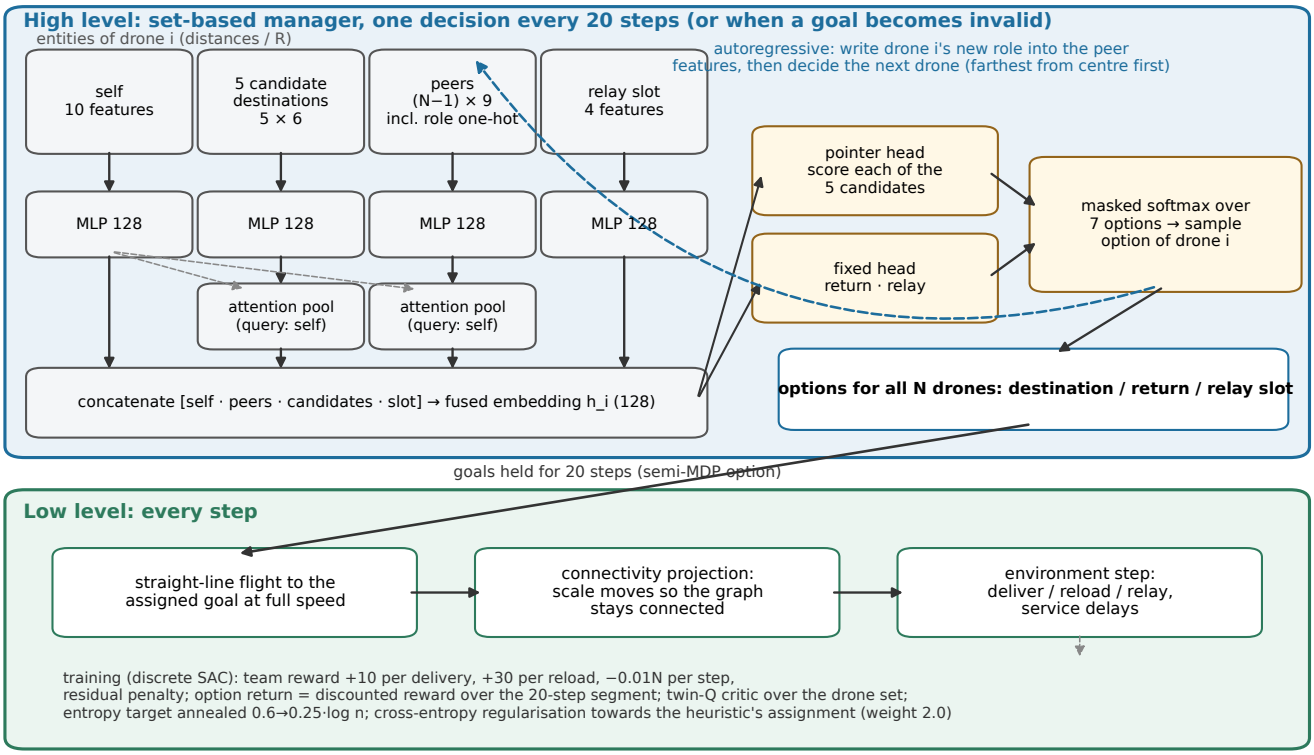


Figure 3: The hierarchical learned controller. High level: the set-based manager encodes each drone from four kinds of entities, pools the peer and candidate sets with attention queried by the drone's own embedding, and scores the five candidates with a pointer head and return/relay with a fixed head; drones are decided one at a time, farthest from the centre first, and each decision is written into the peer features seen by the next drone. Low level, every step: straight-line flight to the assigned goal, the connectivity projection, and the environment step. Options are held for 20 steps.

Procedure 2. Learned manager at a decision point (inference).

1. Build the entity observation of every drone from the fleet state; compute the validity mask of the seven options for every drone (Section 4.1).
2. Order the drones by distance from the centre, farthest first.
3. For each drone i in that order: encode all entities with the current peer roles, compute the seven masked logits of drone i, sample its option from the softmax (sampling, not argmax, is the deployed inference), and write the chosen role (deliver, return or relay) into the peer role one-hot of drone i as seen by the drones not yet decided.
4. Return the N options; the environment converts them into goals and assigns relay slots.

Baselines. Four learning-free baselines establish that the problem is not solved by naive assignment. All use the shared interface of Section 4.1 and are evaluated at the same decision points. *Greedy, no relay* is step 2 of Procedure 1 alone: every drone with cargo flies to its nearest unserved destination and every empty drone returns; nobody ever relays, so the chain is maintained only by the projection, which stalls the fleet. *One (two) fixed relays* assigns the first one (two) drones of the fleet to relay permanently and runs the greedy rule on the others; the relay count is fixed in advance instead of adapting to the lead's distance. *Geometric rule, no escape* is Procedure 1 without step 6. All learning-free controllers are deterministic except for the random tie-breaking of the projection.

Reference model. Small instances are solved with the lattice model of Section 3.1 using Gurobi 13 [26] with six threads; the time limits are given with the results in Section 6.2. Every returned plan is replayed in the continuous simulator.

Paired protocol. Averaging only the episodes each method completes favours the method that gives up on hard instances, and a single long stall moves a 40-rollout mean by over 100 steps. All comparisons between the two controllers are therefore *paired*: both run on the same instance, the difference is taken over instances both complete, and we report the median paired difference, the win–loss count and a Wilcoxon signed-rank p-value. Inference samples from the policy; deterministic argmax inference has the same median but a far heavier tail. The standard configuration uses 480 rollouts (60 seeds, 8 repetitions, two independent measurement runs), the 3/30 and 5/50 configurations 200 (40 seeds, 5 repetitions) and the remaining configurations 80 (40 seeds, 2 repetitions). Repetitions differ because the projection's tie breaking is random. We found 40 rollouts too few to support any claim about tails.

6. Results

6.1. RQ1: Completion under the hard constraint

Connectivity was never violated in any training episode or evaluation rollout, as the projection guarantees. Table 2 reports

Table 2: Completion within the 10,000-step budget. Learning-free baselines on 120 (standard) or 40 (unseen) rollouts; the two proposed controllers on the paired rollout counts of Section 5.

Method	4/50 standard	3/30	5/50
Greedy, no relay	0%	0%	25%
One fixed relay	0%	2.5%	40%
Two fixed relays	3.3%	52.5%	65%
Rule, no escape	61.7%	95%	92.5%
Rule + escape (proposed)	473/480	200/200	191/200
Learned (proposed)	**480/480**	**200/200**	**200/200**
	3/20	4/80	6/80
Rule + escape (proposed)	80/80	80/80	79/80
Learned (proposed)	**80/80**	**80/80**	**80/80**
	8/150	10/300	
Rule + escape (proposed)	80/80	80/80	
Learned (proposed)	**80/80**	**80/80**	

completion within the 10,000-step budget. The learned policy completes every instance in every configuration. The heuristic with escape completes 98–100%; its misses concentrate where drones are plentiful, where a long chain that is not needed still gets stationed and stalls. Without the escape mechanism the rule completes 61.7% of standard instances, and the naive baselines complete almost none: the greedy assignment pulls the fleet apart, and fixed relay counts are wrong for most instances because the number of relays a destination needs varies with its distance.

6.2. RQ2: Distance to the lattice reference plan

Table 3 and Figure 4 compare the two controllers with the lattice reference on 34 instances with three drones: ten each for $M \in \{4, 6, 8\}$ and four for $M = 10$, with time limits of 30, 60, 120 and 120 minutes and six solver threads. Three counts describe what the solver achieved on each instance. A *plan* is a feasible solution of the lattice model returned within the time limit; such a plan is then *replayed* in the continuous simulator by issuing its node sequence as goals, and we count it as replayed only if the replay serves every destination (one plan for $M = 8$ delivered seven of eight destinations on replay, because the continuous projection and the lattice's one-period moves do not coincide exactly, and is counted as found but not replayed); a plan is *optimal* if the solver closed the optimality gap on the lattice model. The replayed makespan is the number we compare against.

Three observations follow. First, the solver's reach ends between six and eight destinations: it returns plans for 9 of 10 instances with four destinations (four of them certified optimal on the lattice), 8 of 10 with six, 3 of 10 with eight, and none of the four with ten within two hours. The bottleneck is feasibility rather than optimality; the remaining gaps on returned plans are 12–36%. Second, where a replayed plan exists both controllers are close to it: the heuristic's makespan averages 1.00, 1.07 and 1.07 times the plan for four, six and eight destinations, and the learned policy's per-instance median averages 0.94, 0.99 and 0.91 times the plan. Both are sometimes faster than the plan, including on instances where the lattice optimum is certified, which is not a contradiction: the lattice restricts where a drone may be, so its optimum is an upper bound on the continuous optimum and a continuous controller can beat it. Third, the learned policy's samples are wide. Its median is at or below the heuristic on most instances, but its range over 30 samples spans up to a factor of five on a single instance (1,142 against a median of 206 on one $M = 8$ instance), which is the same tail behaviour as in Section 6.3. The comparison in RQ3 is therefore against a heuristic that is within a few percent of the best plan an exact method can produce at this scale, and the reference stops being available exactly where the controllers are needed.

6.3. RQ3: Performance across environments

Fleet size divides the configuration space. Table 4 and Figure 5 give the paired comparison across the nine configurations. With three drones the heuristic is faster by 39 and 30 steps in paired median with 20 and 30 destinations. With four drones the two controllers tie: on the standard configuration the median paired difference over 480 rollouts is +2 steps with 234 wins against 239 ($p = 0.52$), and on 4/80 the learned policy is 38 steps faster. With five or more drones the learned policy is faster, by 62 steps at five drones growing to 178 at ten and 76 at twelve (70 wins against 10). The boundary therefore lies at four drones, and the advantage persists beyond the largest fleet we measured, although the monotone growth stops between ten and twelve. We have not isolated the cause of the narrowing at 12/400 and offer three conjectures. First, both controllers maintain a single chain to one lead, so with twelve drones most of the fleet is delivering near the centre regardless of the controller and the decisions that differ between them concern a shrinking share of the work. Second, with 400 destinations a drone's five candidates are all close to it, so the choice among them matters less and the learned policy's advantage, which lies in choosing targets and relays jointly, has less to act on. Third, the manager's hop-depth and slot features are normalised by a constant of eight and exceed one for larger fleets, and twelve sequential autoregressive decisions compound sampling noise; both take the policy farther from its training distribution. Which of these dominates would require fleets beyond twelve drones to settle. An earlier measurement against the heuristic's pre-tuning escape parameters gave the same picture (+40, +0, −54, −170 on 3/30, 4/50, 5/50 and 10/300), so the boundary does not depend on the tuning.

Figure 6 shows the full distributions on the three main configurations: with three drones the heuristic's curve lies to the left, with four the curves coincide, and with five the learned policy's curve lies to the left and reaches completion earlier.

Generalisation from small fleets. The learned policy never saw more than four drones or more than 80 destinations in training, yet it completes every instance with 5 to 12 drones and 50 to 400 destinations and is faster than the heuristic on all of them. Two properties carry this. The set-based encoder has no parameter that depends on N or M, so larger fleets are simply larger sets; and training samples the fleet size and depot afresh every episode. Table 5 separates the two by running the ablation policy that was trained on the fixed 4/50 configuration alone (no

Table 3: Lattice MILP reference versus the proposed controllers on instances with three drones (makespan in steps, means over instances). "plan / replayed / optimal" counts instances where the solver returned a plan, where that plan replayed to completion in the simulator, and where the lattice optimum was certified (gap 0); "gap" is the mean remaining optimality gap on returned plans and "min" the mean solver time. The learned column gives the mean of per-instance medians over 30 samples and the overall min–max. The ratio column divides the heuristic and the learned median by the replayed plan, averaged over replayed instances.

M	inst.	plan / replayed / optimal	gap	min	MILP replay	heuristic	learned median (range)	ratio h / l
4	10	9 / 9 / 4	12%	24	112	114	109 (67–250)	1.00 / 0.94
6	10	8 / 8 / 0	34%	60	149	159	150 (103–415)	1.07 / 0.99
8	10	3 / 2 / 0	36%	120	162	188	171 (119–1142)	1.07 / 0.91
10	4	0 / 0 / 0	—	120	—	224	236 (183–427)	—

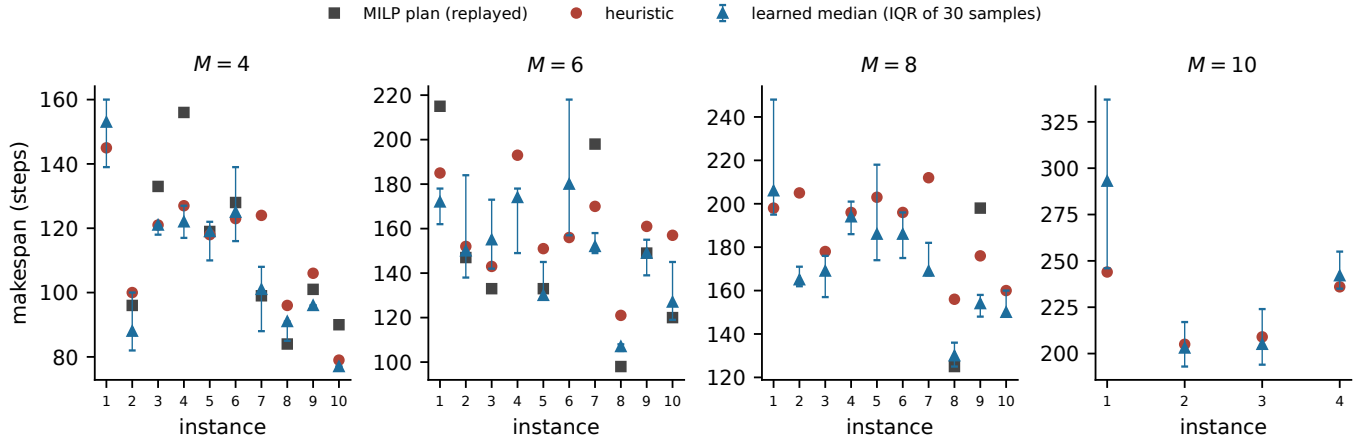


Figure 4: Per-instance makespan on the reference instances with three drones: replayed MILP plan (square, absent where no plan was returned or the replay did not complete), heuristic (circle), and the learned policy's median with the interquartile range of 30 samples (triangle).

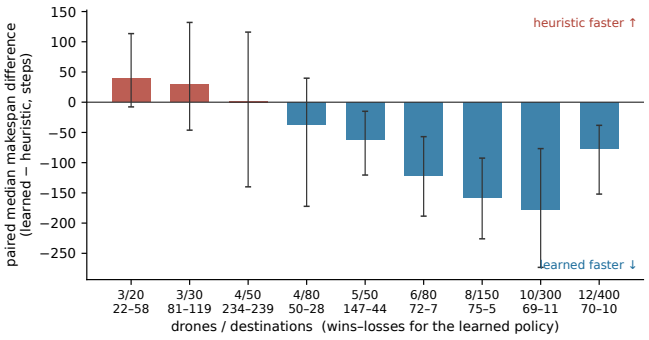


Figure 5: Paired median makespan difference (learned minus heuristic) by configuration, with the interquartile range of the paired differences as error bars and the learned policy's wins–losses under each label. Negative values favour the learned policy; the boundary lies at four drones.

configuration randomisation) on the same large-fleet instances without retraining. That policy also completes every instance from 5 to 10 drones, so completion generalises through the architecture alone; but it is slower than the randomised policy on every configuration (a tie against the heuristic at 5/50 against −62, and −51 to −174 against −122 to −178 beyond), so speed on large fleets is learned from the variety of small ones. The same ablation loses 646 steps on 3/30 (Table 9), so randomisation is what makes the small-fleet side usable at all. Both policies were trained with at most four drones; the hop-depth and slot features are normalised by a constant of eight, so fleets of ten and twelve drones also take the encoder outside its training range of feature values, and the policy still completes them.

Tails follow the same split. Where drones are scarce the learned policy's worst case is three to four times the heuristic's (maximum 5,395 versus 1,447 steps on 3/30); where they are plentiful it is lighter (1,101 versus 1,734 on 5/50, 1,365 versus 1,753 on 10/300). On the standard configuration over 480 rollouts against the pre-tuning heuristic the learned policy exceeded 1,500 steps five times and the heuristic fourteen times.

Mechanism. The two controllers differ in one decision. On 3/30 the heuristic stations 1.39 relays on average and the learned policy 0.78; the learned fleet therefore spends 27% of drone-steps stalled by the projection against 8% for the heuristic. Overriding only the learned policy's relay assignment with the heuristic's, at inference, recovers the full 44-step gap; matching only the *number* of relays recovers 14 steps of median but removes the tail (maximum from 4,904 to 1,083). The identity of the relaying drone drives the median and the count drives the tail. The information needed to make the heuristic's choice is present in the learned policy's observation: a two-layer classifier predicts the heuristic's relay assignment from the 41 observed features with 85% accuracy, and behaviour cloning of the full rule reaches 95% on relay decisions. When drones are plentiful the opposite happens: the heuristic keeps stationing a chain that the geometry no longer needs, while the learned pol-

Table 4: Paired comparison on identical instances. Positive differences mean the learned policy is slower than the heuristic. p from the Wilcoxon signed-rank test on both-completed pairs.

N/M	rollouts	complete h/l	median heuristic / learned	paired median	win–loss	p
3/20	80	80/80	364 / 420	+39	22–58	$< 10^{-4}$
3/30	200	200/200	572 / 618	+30	81–119	3.4e-04
4/50 (standard)	480	473/480	878 / 878	+2	234–239	0.521
4/80	80	80/80	1,120 / 1,036	−38	50–28	0.006
5/50	200	191/200	588 / 534	−62	147–44	$< 10^{-4}$
6/80	80	79/80	737 / 614	−122	72–7	$< 10^{-4}$
8/150	80	80/80	916 / 722	−158	75–5	$< 10^{-4}$
10/300	80	80/80	1,216 / 1,052	−178	69–11	$< 10^{-4}$
12/400	80	80/80	1,199 / 1,108	−76	70–10	$< 10^{-4}$

Table 5: Generalisation to fleets larger than any seen in training, without retraining. Cells give the learned policy's completions and the paired median difference against the heuristic. The proposed policy is trained on $N \in \{3, 4\}$ with random depots; the comparison policy is the ablation trained on the fixed 4/50 configuration only (two seeds).

N/M	trained on $N \in \{3,4\}$, $M \in [20, 80]$	trained on 4/50 only, seed 1	seed 2
5/50	200/200 / −62	80/80 / +7	80/80 / +44
6/80	80/80 / −122	80/80 / −56	80/80 / −51
8/150	80/80 / −158	80/80 / −129	80/80 / −110
10/300	80/80 / −178	80/80 / −152	80/80 / −174

icy exploits the slack and, with its sampled inference, escapes the stalls that the deterministic rule repeats; this is also where the heuristic loses its few completions.

Why the scarce-fleet deficit resists policy-side fixes. Table 6 lists eleven interventions, each trained with four or two seeds under the same recipe except for the stated change and evaluated with the same paired protocol, that attempt to make the learned policy relay like the heuristic when drones are scarce. The variants are, in the numbering of the table: (V1) the cross-entropy regularisation towards the heuristic is weighted three times higher for drones whose heuristic option is relay; (V2) each drone on the communication path of a delivering teammate receives an individual reward (2.0 or 6.0) at that delivery and the critic is trained on per-drone returns; (V3) checkpoint selection scores half of its held-out instances on the 3/30 configuration instead of only on 4/50, eight seeds; (V4) as V2 but zero-sum, the delivering drone losing what its relays gain; (V5) the critic is conditioned on the teammates' chosen roles (a joint-action critic) with the teammates' next roles sampled from the actor in the target; (V6) the autoregressive order is reversed to decide the drone nearest the centre first; (V7) all goals are invalidated after 60 consecutive partial-deadlock steps so that the manager re-decides, in training and evaluation; (V8) the mean over per-drone action values in the critic is replaced by state-dependent monotone weights that sum to one with a floor of one half; (V9) each drone's option is stored as its own single-agent transition, chained inside a decision point with discount one, so that no decomposition of the team return is needed; (V10) the manager is initialised by behaviour cloning of the heuristic on 40,000 decisions (92.5% accuracy, 95% on relay decisions) and then fine-tuned, with and without a 50-episode critic warm-up; (V11) a per-step penalty proportional to the estimated arrival time of the bottleneck destination, with the estimate divided by each drone's realised-movement ratio so that blocked drones count as slow. None improves the 3/30 median and several hurt the configurations where the learned policy wins. The pattern is informative. Adding reward for relaying turns the means into the end: the policy relays for its own sake and 5/50 reverses from −54 to +60. Every route to per-drone credit is blocked for a different reason: a joint-action critic inherits the variance of a sampled teammate action in its target; a monotonic mixing network either diverges when its weights are free or collapses to the mean when they are floored; a sequential single-agent reformulation removes the decomposition but leaves an undiscounted credit chain with no contraction. Initialising the policy by behaviour cloning of the rule, with 95% relay accuracy, is erased within 400 episodes: once the actor is updated, relay usage falls to the level of a randomly initialised policy within 50 episodes and stays there. A penalty on the bottleneck destination's estimated arrival makes the policy reduce the *average* rather than the maximum. We conclude that, within this policy structure and objective, under-relaying is the learner's fixed point, not an artefact of credit, information or initialisation.

Sensitivity to the connectivity enforcement model. The projection of Section 4.1 scales a blocked displacement along its own direction, so a drone pulling on the chain effectively stops. We implemented an alternative that clamps the target position to the nearest point of the communication disc around the binding neighbour, which removes only the outward component and keeps the tangential one (a drone can still circle at constant range). Both controllers use the same projection in each comparison, and connectivity is still never violated. These measurements predate the heuristic tuning of Section 6.4 and use $k_{\text{stall}} = 60$, $h_{\text{hold}} = 40$ for both projections, so they are internally

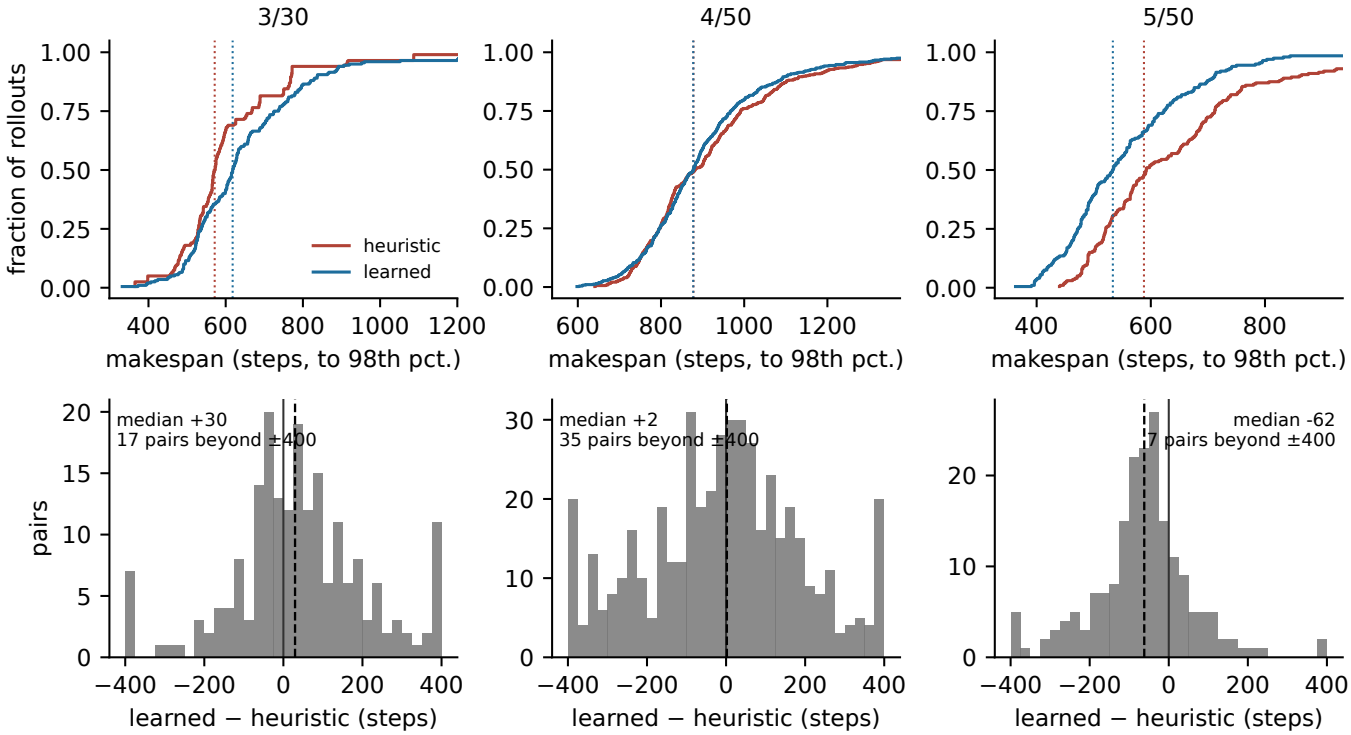


Figure 6: Top: cumulative distribution of makespan on 3/30, 4/50 and 5/50 (dotted lines mark medians; axes cut at the 98th percentile). Bottom: distribution of the paired difference on both-completed instances, clipped to ±400 steps with the number of clipped pairs noted.

consistent. Evaluating the same learned weights under the sliding projection without retraining reduces the 3/30 deficit from +40 to +21; retraining four seeds under it gives +15 to +48 (mean +28) and lowers the learned fleet's stall ratio from 0.27 to 0.17–0.22, with completion still 100%. The heuristic gains little with three drones (median 570 to 571) but gains more than the learned policy with four (866 to 812 against 879 to 850), so the standard-configuration tie becomes +26 in its favour. We therefore keep the radial projection as the primary model and report this as a sensitivity result: about half of the three-drone deficit is a property of how the constraint is enforced rather than of the learner.

Communication radius. Both controllers were run without retraining at $R = 250$ m and $R = 350$ m (Table 7, Figure 7); the learned policy normalises distances by R, and the instance generator rescales the reachable region $0.9NR$ with R, so both controllers see the same instances at each radius. Completion transfers: the learned policy completes every instance at both radii, while the heuristic loses 18 of 200 instances on 5/50 at the shorter radius. Speed shows two effects. The first is the regime shift seen with fleet size: at $R = 250$ m a destination needs more hops, the fleet is effectively scarcer, and every configuration moves towards the heuristic (+77 on 3/30, +68 on 4/50, and the 5/50 advantage shrinks to a tie at −16, $p = 0.33$); at $R = 350$ m the fleet is effectively more plentiful and the learned policy is faster already with four drones (−40 on 4/50). The second is a training-radius effect: on 3/30 and 5/50 the learned policy's margin is best at the radius it was trained on and worse on both sides (+30 at 300 m against +77 and +52; −62 against −16 and −42), so the normalisation by R transfers completion but not the full speed. The regime variable is therefore the chain demand, the hops a destination needs relative to the drones available, rather than the fleet size alone, and a deployment at a radius far from the training radius should retrain or at least re-evaluate the boundary.

6.4. RQ4: Are the components necessary?

Heuristic. Table 8 grids $k_{\text{stall}} \in \{30, 60, 90, 120\}$, $h_{\text{hold}} \in \{20, 40, 80\}$ and the two escape modes, plus no escape, on 3/30, 4/50 and 5/50 with 40 tuning seeds each, and ranks settings by total completions first and the sum of median makespans second; this criterion was fixed before the measurement. The escape mechanism is necessary: without it the rule completes 26 of 40 standard instances. The mode matters more than the timing: every lead-swap setting completes 117–120 of 120 instances, while retreat settings complete 100–117 because slackening the chain does not change the assignment that caused the stall. Within the lead-swap family the medians differ by at most 40 steps summed over three configurations, which is within the noise of 40 instances; the selected setting ($k_{\text{stall}} = 120$, $h_{\text{hold}} = 20$) is the only one that completes all 120 and also has the lowest median sum. The adaptive relay count is necessary as well: fixing one or two relays completes 0–65% (Table 2).

Learned policy. Table 9 removes one ingredient at a time, retrains two seeds and evaluates on the three main configurations;

Table 6: Interventions that did not remove the scarce-fleet deficit, numbered as in the text. Paired median against the heuristic (positive is slower); the proposed recipe is the reference row; two values are two seeds and a range is the span over seeds or doses. These runs were evaluated against the heuristic with its pre-tuning escape parameters ($k_{\text{stall}} = 60$, $h_{\text{hold}} = 40$), so the column values are comparable to the reference row of this table but not numerically identical to Table 4, which uses the tuned heuristic.

#	Intervention (what changed)	3/30	5/50	4/50	Mechanism of failure
—	Proposed recipe (reference)	+40	−54	+0	—
V1	Relay-weighted rule regularisation (×3 on relay)	+54, +56	−44, −19	+41, +54	relay-specific regularisation harms
V2	Per-drone relay credit (2.0 / 6.0)	+36 to +82	+60 to +106	+39 to +131	relaying becomes the goal
V3	Checkpoint selection half on 3/30 (8 seeds)	+58	−42	+10	improves only the standard config.
V4	Zero-sum credit redistribution	+91, +126	+98, +172	+146, +180	same, larger
V5	Joint-action critic	+89	−27	+58	target variance from sampled teammates
V6	Nearest-first decision order	+46, +100	−43, −38	+4, +106	order is not the cause
V7	Re-decision after 60 stalled steps	+82, +95	−38, −34	+17, +55	irregular option lengths
V8	Monotone mixing critic (floor 0.5)	+145	+5	+139	divergence vs. credit on one axis
V9	Sequential single-agent credit	+142	+17	+116	no contraction within a decision
V10	Behaviour-cloning initialisation (± warm-up)	+42 to +69	−42 to −20	−4 to +40	erased within 400 episodes
V11	Bottleneck-arrival penalty (0.15 / 0.25)	+105	−8	+81	reduces the mean, not the maximum

Table 7: Transfer to other communication radii without retraining. Cells give completions (heuristic/learned) and the paired median difference with win–loss and p; $R = 300$ m is the training radius.

N/M	$R = 250$ m	$R = 300$ m	$R = 350$ m
3/30	200/200 / +77 (45–154, $< 10^{-4}$)	200/200 / +30 (81–119, 3.4e-04)	200/200 / +52 (58–141, $< 10^{-4}$)
4/50	118/120 / +68 (40–77, $< 10^{-4}$)	473/480 / +2 (234–239, 0.521)	119/120 / −40 (77–42, 1.1e-04)
5/50	182/200 / −16 (99–83, 0.327)	191/200 / −62 (147–44, $< 10^{-4}$)	200/200 / −42 (152–46, $< 10^{-4}$)

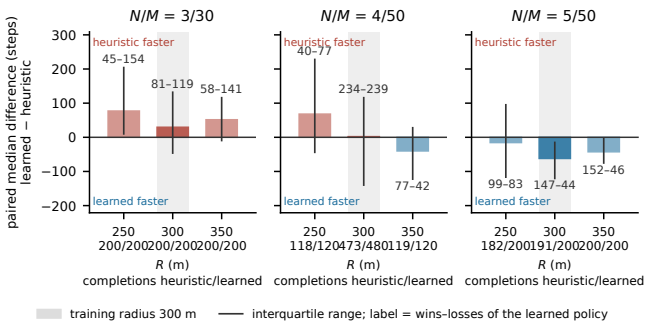


Figure 7: Transfer to other communication radii without retraining, one panel per configuration. Bars are the paired median difference (learned − heuristic, steps): a bar above zero (red) means the heuristic was faster, below zero (blue) the learned policy; whiskers are the interquartile range of the paired differences, the label at each bar is the learned policy's wins–losses, and the completions (heuristic/learned) are printed under the radius. The shaded column is the training radius. Reading left to right within a panel: a shorter radius moves every configuration towards the heuristic, and on 3/30 and 5/50 the learned policy's margin is best at the radius it was trained on.

values are makespan differences from the proposed recipe. Four independent trainings of the same recipe span 919 to 979 steps on the standard configuration, so a two-seed mean is meaningful only beyond 70–80 steps. Rule regularisation and configuration randomisation are necessary; entropy annealing is the single largest contributor on the standard configuration; autoregressive decisions help with four drones and more; the residual penalty is neutral. A manager with a fixed-size MLP encoder instead of the set encoder, trained in an earlier version on the standard configuration only, completed 38% of standard instances and could not be run on other fleet sizes at all.

6.5. *Inference cost*

Table 10 gives the latency of one manager decision on a CPU. The heuristic costs 0.05–0.12 ms; the learned policy 1.3–2.5 ms without autoregression and 2.7–7.4 ms with it, growing linearly with the fleet because each drone is a sequential network call. Over a 900-step episode the learned policy's cumulative decision time is 0.16 s. The lattice MILP takes 600 to 5,400 seconds on instances with four to ten destinations and does not return a plan for ten.

7. Discussion

Deployment. The two controllers are complementary and the boundary is sharp, so a fleet operator can choose by fleet size: the heuristic with three drones, the learned policy with five or more, where it is both faster and more reliable; with four the two tie and the learned policy completes more. Both decide in milliseconds and both keep the fleet connected by construction.

What the negative results mean. Ten interventions that change how the learner is credited, informed, initialised or rewarded leave the scarce-fleet deficit in place, and a change to the environment's enforcement model removes half of it. The deficit is therefore best understood as an interaction between the learner's fixed point under this objective and the geometry of the constraint, not as a shortcoming that more training or a better critic would fix. For a practitioner the actionable consequence is the one above: pick the controller by regime.

Limitations. The learned policy was trained at a single communication radius; it transfers to radii of 250 and 350 m in completion but its speed ranking against the heuristic shifts with the

Table 8: Heuristic ablation on tuning seeds disjoint from the evaluation seeds (40 instances per configuration). Cells give completed instances and the median makespan over completed instances; the last column sums completions over the three configurations.

mode	k_{stall} / h_{hold}	3/30	4/50	5/50	completions
none	—	39/40 / 561	26/40 / 805	36/40 / 613	101/120
lead swap	30 / 20	40/40 / 566	38/40 / 827	40/40 / 630	118/120
lead swap	30 / 40	40/40 / 560	39/40 / 847	40/40 / 625	119/120
lead swap	30 / 80	40/40 / 565	39/40 / 892	38/40 / 628	117/120
lead swap	60 / 20	40/40 / 562	40/40 / 879	39/40 / 631	119/120
lead swap	60 / 40	40/40 / 561	39/40 / 852	40/40 / 624	119/120
lead swap	60 / 80	40/40 / 561	38/40 / 838	40/40 / 636	118/120
lead swap	90 / 20	40/40 / 562	39/40 / 865	40/40 / 624	119/120
lead swap	90 / 40	40/40 / 565	39/40 / 887	38/40 / 594	117/120
lead swap	90 / 80	40/40 / 560	39/40 / 891	40/40 / 637	119/120
lead swap	**120 / 20**	40/40 / 568	40/40 / 826	40/40 / 603	**120/120**
lead swap	120 / 40	40/40 / 565	39/40 / 869	39/40 / 614	118/120
lead swap	120 / 80	40/40 / 558	38/40 / 834	40/40 / 621	118/120
retreat	30 / 20	37/40 / 560	38/40 / 810	37/40 / 605	112/120
retreat	30 / 40	38/40 / 558	33/40 / 824	38/40 / 637	109/120
retreat	30 / 80	37/40 / 557	27/40 / 812	38/40 / 608	102/120
retreat	60 / 20	39/40 / 566	36/40 / 796	40/40 / 622	115/120
retreat	60 / 40	39/40 / 550	39/40 / 833	39/40 / 613	117/120
retreat	60 / 80	39/40 / 560	25/40 / 823	36/40 / 634	100/120
retreat	90 / 20	39/40 / 557	38/40 / 829	39/40 / 606	116/120
retreat	90 / 40	39/40 / 563	39/40 / 855	39/40 / 616	117/120
retreat	90 / 80	39/40 / 566	35/40 / 838	40/40 / 641	114/120
retreat	120 / 20	39/40 / 556	39/40 / 869	38/40 / 626	116/120
retreat	120 / 40	39/40 / 558	36/40 / 873	38/40 / 608	113/120
retreat	120 / 80	39/40 / 556	37/40 / 863	40/40 / 616	116/120

Table 9: Ablation of the learned policy: makespan difference from the proposed recipe over completed episodes, two seeds. Positive means the removed component was needed. Differences below 70–80 steps are within seed variance.

Removed component	4/50	3/30	5/50
Rule regularisation	+320	+408	+92
Configuration randomisation	−44	+646	+68
Entropy target annealing	+128	+19	+61
Autoregressive decisions	+94	−46	+33
Residual penalty	−28	−37	+8

radius (Section 6.3), and radii outside this range were not tested. The environment is planar, ignores wind, battery and collisions, and uses a disc communication model; real links are not discs. The MILP reference is a lattice restriction with a remaining optimality gap, not an exact optimum. The heuristic's novelty lies in the escape mechanism and the adaptive relay count rather than in the chain geometry itself.

Methodological notes. Three practices mattered for the conclusions. Completed-episode averages favour the controller that gives up, so comparisons must be paired. Forty rollouts cannot support claims about tails; several of our own interim conclusions reversed when the sample grew. And a pre-check that uses a different metric from the final judgement will pass interventions that the final judgement rejects.

8. Conclusion

We proposed two controllers for connectivity-constrained drone delivery, a geometric chain-relay heuristic with deadlock escape and a fleet-size-agnostic learned policy, both acting through a projection that keeps the fleet connected at every step. Under a paired protocol across fleets of three to twelve drones the learned policy completes every instance and is faster by 38 to 178 steps once more than four drones are available, although it was trained on three and four, while the heuristic is faster by 30–39 steps with three. The scarce-fleet deficit is one decision, which drone relays; it resists ten policy-side interventions and halves when the enforcement model is changed. Both controllers are three orders of magnitude faster than exact optimisation on a lattice, which stops returning plans at ten destinations. The natural next step is a policy whose output is a team assignment rather than a set of per-drone options, so that the relay identity becomes a coordinate the critic can value directly.

Acknowledgements

[To be added.]

Data and code availability

Code, trained weights, instance generators and the paired-comparison records are available at `https://github.com/koptimizer/relay_drone_network`.

Table 10: Latency of one manager decision (milliseconds, CPU) and the cumulative decision time of a 900-step episode.

Controller	$N = 3$	$N = 4$	$N = 6$	$N = 8$	900 steps
Geometric rule + escape	0.05	0.08	0.10	0.12	0.003 s
Learned, independent decisions	1.27	1.52	1.94	2.45	0.069 s
Learned, autoregressive (proposed)	2.70	3.52	5.32	7.35	0.158 s

References

[1] C. C. Murray, A. G. Chu, The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery, Transportation Research Part C: Emerging Technologies 54 (2015) 86–109. doi:10.1016/j.trc.2015.03.005.

[2] C. C. Murray, R. Raj, The multiple flying sidekicks traveling salesman problem: Parcel delivery with multiple drones, Transportation Research Part C: Emerging Technologies 110 (2020) 368–398. doi:10.1016/j.trc.2019.11.003.

[3] N. Agatz, P. Bouman, M. Schmidt, Optimization approaches for the traveling salesman problem with drone, Transportation Science 52 (4) (2018) 965–981. doi:10.1287/trsc.2017.0791.

[4] K. Dorling, J. Heinrichs, G. G. Messier, S. Magierowski, Vehicle routing problems for drone delivery, IEEE Transactions on Systems, Man, and Cybernetics: Systems 47 (1) (2017) 70–85.

[5] J. Park, C. Kwon, J. Park, Learn to solve the min-max multiple traveling salesmen problem with reinforcement learning, in: Proceedings of the 22nd International Conference on Autonomous Agents and Multiagent Systems, 2023, pp. 878–886.

[6] Y. Guo, Z. Ren, C. Wang, iMTSP: Solving min-max multiple traveling salesman problem with imperative learning, arXiv:2405.00285 (2024).

[7] Y. Zhu, H. Xu, A. A. Malikopoulos, N. Geroliminis, Dual-mission drone fleets: Delivery with secondary sensing tasks, arXiv:2604.02471 (2026).

[8] Y. Zeng, R. Zhang, T. J. Lim, Throughput maximization for UAV-enabled mobile relaying systems, IEEE Transactions on Communications 64 (12) (2016) 4983–4996. doi:10.1109/TCOMM.2016.2611512.

[9] E. Yanmaz, Positioning aerial relays to maintain connectivity during drone team missions, Ad Hoc Networks 128 (2022) 102800.

[10] R. Ding, J. Chen, W. Wu, J. Liu, F. Gao, X. Shen, Packet routing in dynamic multi-hop UAV relay network: A multi-agent learning approach, IEEE Transactions on Vehicular Technology 71 (9) (2022) 10059–10072.

[11] MUTTO: Multi-agent reinforcement learning for joint trajectory and transmit power optimisation of relay and mission UAVs, Vehicular Communications (2024).
URL https://www.sciencedirect.com/science/article/abs/pii/S1570870524002130

[12] MAEN: Multi-agent aerial relay networking, Proceedings of ACM MobiCom (2024). doi:10.1145/3636534.3694730.

[13] M. Li, Y. Jie, Y. Kong, H. Cheng, Decentralized global connectivity maintenance for multi-robot navigation: A reinforcement learning approach, arXiv:2109.08536 (2021).

[14] J. De Hoog, S. Cameron, A. Visser, Role-based autonomous multi-robot exploration, in: Proceedings of the International Conference on Advanced Cognitive Technologies and Applications, 2009.

[15] K. Cesare, R. Skeele, S.-H. Yoo, Y. Zhang, G. Hollinger, Multi-UAV exploration with limited communication and battery, in: Proceedings of the IEEE International Conference on Robotics and Automation, 2015, pp. 2230–2235. doi:10.1109/ICRA.2015.7139494.

[16] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: Proceedings of the 35th International Conference on Machine Learning, 2018, pp. 1861–1870.

[17] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, I. Mordatch, Multi-agent actor-critic for mixed cooperative-competitive environments, in: Advances in Neural Information Processing Systems 30, 2017, pp. 6379–6390.

[18] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, S. Whiteson, Counterfactual multi-agent policy gradients, in: Proceedings of the 32nd AAAI Conference on Artificial Intelligence, 2018, pp. 2974–2982.

[19] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, T. Graepel, Value-decomposition networks for cooperative multi-agent learning based on team reward, in: Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems, 2018, pp. 2085–2087.

[20] T. Rashid, M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster, S. Whiteson, QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning, in: Proceedings of the 35th International Conference on Machine Learning, 2018, pp. 4295–4304.

[21] R. S. Sutton, D. Precup, S. Singh, Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning, Artificial Intelligence 112 (1–2) (1999) 181–211.

[22] S. Huang, S. Ontañón, A closer look at invalid action masking in policy gradient algorithms, in: Proceedings of the International FLAIRS Conference, Vol. 35, 2022.

[23] G. Dalal, K. Dvijotham, M. Vecerik, T. Hester, C. Paduraru, Y. Tassa, Safe exploration in continuous action spaces, arXiv:1801.08757 (2018).

[24] A. Y. Ng, D. Harada, S. Russell, Policy invariance under reward transformations: Theory and application to reward shaping, in: Proceedings of the 16th International Conference on Machine Learning, 1999, pp. 278–287.

[25] L. R. Ford, D. R. Fulkerson, Flows in Networks, Princeton University Press, Princeton, NJ, 1962.

[26] Gurobi Optimization, LLC, Gurobi optimizer reference manual, version 13, https://www.gurobi.com (2026).